RTL DESIGN AND VERIFICATION OF DIGITAL SECURITY LOCKER USING FSM USING EDA PLAYGROUND

EMPLOYABILITY ENHANCEMENT COURSE

SD0212 RTL DESIGN AND VERIFICATION

PROJECT REPORT

Submitted by

P. PRITHIKA	**312424106128**
SHERLEY KETZIAH	**312424106162**
RANJANA DEVI	**312424106131**
YOGA	**312424106188**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

St. JOSEPH'S INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

OMR, CHENNAI – 600 119

BATCH: 2024-2028

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that the Employability Enhancement Course Project Report titled “RTL Design and Verification of Digital Security Locker using FSM” is the bonafide work of **“P. PRITHIKA (312424106128), SHERLEY KETZIAH (312424106162), RANJANA DEVI (312424106131) and YOGA (312424106188)”**, who completed the training program under my supervision.

SIGNATURE

Dr. R. NIROSHA, M.E., Ph.D.,

SUPERVISOR,

Assistant Professor,

Electronics and Communication Engineering,

St. Joseph’s Institute of Technology,

Old Mamallapuram Road,

Chennai – 600 119.

SIGNATURE

Dr. P.G.V. RAMESH, M.Tech., Ph.D.,

HEAD OF THE DEPARTMENT,

Professor,

Electronics and Communication Engineering,

St. Joseph’s Institute of Technology,

Old Mamallapuram Road,

Chennai – 600 119.

ACKNOWLEDGEMENT

At the outset, I would like to express my sincere gratitude to our beloved Chairman ***Dr. B. BABU MANOHARAN, M.A., M.B.A., Ph.D.,*** for his constant guidance and support.

I would like to express my sincere thanks to our respected Executive Director, ***Mrs. S. JESSIE PRIYA, M. Com.,*** and our Managing Director, ***Mr. B. SASHISEKAR, M.Sc*** for their kind encouragement and blessings.

I express my sincere gratitude and wholehearted thanks to our Principal ***Dr. R. PRISCILLA, M.E., Ph.D.,*** for her encouragement to make this Training a successful one.

I wish to express our truthful thanks and gratitude to our Head of the Department ***Dr. P.G.V. RAMESH, M.Tech., Ph.D.,*** Department of Electronics and Communication Engineering for his constant support and motivation throughout my internship.

I have no words to express my heartfelt thanks to ***Dr. R. NIROSHA, M.E., Ph.D.,*** Assistant Professor for her guidance, support and encouragement in completing the Training and Project within the stipulated time.

ABSTRACT

This project focuses on the RTL design and verification of a **Digital Security Locker using a Finite State Machine (FSM)** on the EDA Playground platform. The locker accepts a 4-bit password from the user, compares it with a stored, parameterised password (4'b1010) and drives three outputs: an *unlock* signal, a *buzzer* alarm and a 2-bit *attempt_count*. The design is modelled as a four-state FSM – IDLE, CHECK, UNLOCKED and LOCKOUT – with an asynchronous active-high reset, which makes the architecture compact, modular and easy to verify. A correct password opens the locker, a wrong password increments the attempt counter, and three wrong attempts push the locker into a lockout state in which the buzzer is turned on and the locker stays locked until it is reset.

At the RTL stage, the Verilog description of the locker was developed and tested with a self-checking testbench. The testbench generates a 100 MHz clock and reset, applies three wrong passwords followed by the correct password after lockout through a reusable task, and prints a PASS or FAIL message by checking attempt_count, buzzer and unlock. Functional verification therefore covered reset behaviour, the counting of wrong attempts, the buzzer alarm, and the case in which the correct password is entered after lockout and must be rejected.

Simulation was carried out on EDA Playground and the waveforms were analysed in EPWave. The waveforms show attempt_count advancing 0 → 1 → 2 → 3, the buzzer asserting on the third wrong attempt, and unlock remaining low even when the correct password is applied after lockout. The results confirm that the RTL meets its functional specification. The verified RTL forms the baseline for the subsequent stages of the RTL-to-GDSII flow such as synthesis, physical design, static timing analysis and layout verification.

CONTENTS

CHAPTER NO.	Title	Page No.
	ABSTRACT	**iv**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vi**
1	**INTRODUCTION**	**1**
	1.1 Overview	
	1.2 Key Steps in the VLSI Design Process	
	1.3 Summary	
2	**RTL DESIGN AND VERIFICATION**	**4**
	2.1 Introduction	
	2.1.1 IC Design	
	2.1.2 IC Manufacturing	
	2.1.3 Post Tape out IC Testing and Validation	
	2.1.4 Integration into devices	
	2.2 RTL Design Flow and Verification	
	2.2.1 Front End Design	
	2.3 Summary	
3	**RESULT AND DISCUSSION**	**7**
	3.1 PROPOSED SYSTEM	7
	3.1.1 Design Specification	7
	3.1.2 RTL Design	10
	3.1.3 RTL Verification	11
	3.1.4 Waveform Generation using EDA PLAYGROUND	13
	3.1.5 Operation Flow, Timing and Coverage Analysis	15
	3.1.6 Position of the Project in the RTL-to-GDSII Flow	17
	3.2 SUMMARY	18
4	**CONCLUSION AND FUTURE SCOPE**	**18**

LIST OF TABLES

Table No.	Name	Page No.
Table 3.1	Digital Security Locker port specifications	7
Table 3.2	FSM state encoding and description	8
Table 3.3	State transition table	8
Table 3.4	Test plan for the Digital Security Locker	12
Table 3.5	Observed behaviour from the simulation waveform	14
Table 3.6	State and transition coverage of the testbench	16
Table 3.7	Estimated hardware resources	16
Table 3.8	Proposed additional test scenarios	17

LIST OF FIGURES

Figure No.	Name	Page No.
Figure 3.1	State diagram of the Digital Security Locker FSM	8
Figure 3.2	Block diagram of the Digital Security Locker	9
Figure 3.3	Gate-level circuit of the 4-bit password comparator (PASSWORD = 1010)	9
Figure 3.4	Circuit of the 2-bit attempt counter (incrementer and decoder)	10
Figure 3.5	Console output of the self-checking testbench	14
Figure 3.6	EPWave waveform generation of the Digital Security Locker	14
Figure 3.7	Flow chart of the Digital Security Locker operation	15
Figure 3.8	Timing diagram of one wrong-password submission	16
Figure 3.9	Stages of the RTL-to-GDSII flow (shaded: completed in this project)	17

CHAPTER 1

INTRODUCTION

1.1 Overview

What is VLSI?

VLSI, or Very Large-Scale Integration, is a cutting-edge technology in the field of electronics that involves the design and fabrication of integrated circuits (ICs) by integrating millions or even billions of transistors onto a single silicon chip. These chips serve as the "brains" of electronic devices, enabling a wide range of functionalities from complex calculations to high-speed data processing.

The Revolution of VLSI Technology

VLSI technology has transformed the way we create electronic circuits. Traditionally, electronic devices were built using bulky circuits composed of individual components like resistors, capacitors, and discrete transistors. VLSI replaces these cumbersome designs with highly compact and efficient chips, often no larger than a coin. This miniaturization has led to the development of powerful, portable, and energy-efficient devices that are integral to modern life.

Why VLSI is Important

Miniaturization of Devices:

Impact: VLSI allows for the creation of incredibly small and compact electronic devices. This miniaturization is what makes modern gadgets like smartphones, laptops, and smartwatches possible.

Example: A smartphone today has more computing power than the room-sized computers of the past, all thanks to VLSI technology.

Improved Performance:

Impact: By integrating more transistors onto a single chip, VLSI enhances the performance of electronic devices. This means faster processing speeds, better data handling, and overall improved functionality.

Example: High-performance gaming consoles and graphics cards rely on VLSI to deliver seamless and immersive gaming experiences.

Reduced Power Consumption:

Impact: VLSI chips are designed to be energy-efficient, reducing the power consumption of electronic devices. This makes devices not only faster but also more environmentally friendly.

Example: Energy-efficient processors in laptops and smartphones extend battery life, allowing users to stay connected for longer periods.

Cost Efficiency:

Impact: The mass production of VLSI chips reduces manufacturing costs significantly. This cost efficiency makes advanced electronics more affordable for consumers.

Example: The affordability of smartphones and other consumer electronics is largely due to the cost-effective production of VLSI chips.

1.2 Key Steps in the VLSI Design Process

Specification:

Purpose: Engineers define the functional requirements of the chip. This includes deciding what the chip should do, such as powering a smartphone, processing video, or managing data in a server.

Output: A detailed specification document outlining the chip's intended capabilities and performance metrics.

Design:

Purpose: A blueprint of the chip is created using specialized software tools. This design phase includes both the behavioural and structural aspects of the chip.

Process: Engineers use hardware description languages (HDLs) like Verilog or VHDL to describe the chip's functionality. This high-level design is then synthesized into a gate-level netlist.

Output: A comprehensive design that includes the layout and interconnections of all components on the chip.

Fabrication:

Purpose: The chip is physically built in a fabrication facility, also known as a foundry. This process involves layering and connecting millions of tiny components to create a functional integrated circuit.

Process: The design is translated into a series of photolithographic masks, which are used to pattern the layers of the chip. Advanced manufacturing techniques ensure precision and accuracy.

Output: A fully fabricated chip ready for testing and integration into electronic devices.

Testing: *Purpose*: The final chip is rigorously tested to ensure it meets the specified performance and reliability standards. This includes functional testing, electrical testing, and environmental testing.

Process: Automated test equipment (ATE) is used to simulate various operating conditions and verify the chip's functionality.

Output: A certified chip that is ready for mass production and integration into end-user devices. Applications of VLSI

VLSI chips are used in a wide range of devices and industries, such as

1. Consumer Electronics:

- Smartphones (processors, memory, and sensors)
- Laptops and PCs (CPUs, GPUs, and RAM)
- Smart TVs and home assistants

2. Automotive Industry:

- Advanced driver-assistance systems (ADAS)
- Engine control units (ECUs) for efficient vehicle performance
- Electric vehicles (EVs) for battery management

3. Medical Devices:

- Wearable health monitors (like fitness trackers)
- Imaging systems (like MRI machines)
- Portable diagnostic tools

4. Aerospace and Defence:

- Satellite communication systems
- Radar and navigation systems
- High-speed data processing for military applications

5. Communication Networks:

- 5G and wireless communication infrastructure
- Routers and modems for internet connectivity

1.3 Summary

VLSI is at the heart of modern technology, powering the devices and systems we use daily. Its ability to integrate countless functions into a small chip makes it indispensable for innovation. From healthcare to entertainment, VLSI enables industries to thrive and evolve, pushing the boundaries of what's possible.

CHAPTER 2

RTL DESIGN AND VERIFICATION

2.1 Introduction

The entire Chip life cycle can be characterized by a few key stages that are briefly discussed below. The focus of this report will be the IC design stage

2.1.1 IC Design:

IC design is where the journey begins. It's the process of conceptualizing, defining, and creating the blueprint for an integrated circuit. It can be broadly divided into:

- **System-Level Design**: Translating user requirements and device functionality into system specifications.
- **RTL Design (Digital)**: Using hardware description languages like Verilog or VHDL to describe the logical behaviour.
- **Analog/Custom Design**: Creating precise transistor-level schematics for analog and mixed-signal components.
- **Verification and Validation**: Ensuring correctness through simulation, synthesis, and formal verification.
- **Physical Design**: Transforming logical designs into layouts for fabrication, including placement, routing, and timing closure.

This step combines creativity and engineering precision to design ICs that are smaller, faster, and energy efficient.

2.1.2 IC Manufacturing:

Once the design is finalized, the IC undergoes fabrication in highly specialized semiconductor foundries. This process includes:

- **Wafer Fabrication**: Creating wafers with multiple chips using photolithography, doping, etching, and deposition techniques.
- **Material Science**: Using advanced materials like silicon, gallium arsenide, or even emerging technologies like graphene.
- **Moore's Law in Action**: Shrinking transistor sizes for each generation to pack billions of transistors onto a single chip.

- **Packaging**: Encasing the fabricated chip in protective materials, ensuring electrical connections and heat dissipation.

This is a highly capital-intensive stage, requiring precision and expertise to achieve high yield and quality.

2.1.3 Post Tape-out IC Testing and Validation:

After tape-out (finalizing the layout for fabrication), rigorous testing and validation are performed to ensure the IC functions as intended:

- **Wafer Testing**: Probing individual chips on the wafer to test functionality before cutting them.
- **Package-Level Testing**: Checking electrical, thermal, and mechanical properties after packaging.
- **Burn-In Testing**: Running the chip at elevated temperatures and voltages to screen for early-life failures.
- **Field Tests and Debugging**: Evaluating performance in real-world conditions and fine-tuning as needed.

This stage ensures the IC is reliable and meets the required performance standards before it enters the market.

2.1.4 Integration into Devices:

Once validated, these ICs are integrated into smart devices such as smartphones, IoT devices, and autonomous vehicles. The entire process—from design to post-tape-out testing—is a marvel of modern engineering, pushing the boundaries of what's possible in technology.

The IC Design is the focus of the work we are concerned with here. To begin, we first understand the classification of IC design flows.

1. RTL to GDSII Flow
2. Custom / AMS Design flow

2.2 RTL Design and Verification:

RTL design and verification is a crucial step in the digital IC design flow. In this stage, designers write RTL (Register Transfer Level) code in languages like Verilog or VHDL to describe the digital circuit's behavior. The RTL code is then simulated and verified using EDA tools like VCS, Xcelium, or Riviera-PRO to ensure that the design meets the required specifications.

A testbench is created to apply inputs, generate clock and reset signals, and check results. The simulation waveforms are analyzed to identify and fix any issues, ensuring the design's functionality and performance. By verifying the RTL design thoroughly, designers can catch and fix errors early on, reducing the risk of costly rework later in the design cycle.

2.2.1 Front-End Design

Front-End Design involves defining the functionality and behaviour of the integrated circuit. It is focused on the logical and architectural aspects of the design. Here's a deeper look at its key components:

1. **Design Entry (RTL Coding):**
    - Specifying the architecture and functionality of the 8x8 sequential multiplier to meet its intended application needs in digital processing.
    - Defines how different components (e.g., processors, memory, communication interfaces) interact with each other.
2. **Test Bench Creation:**
    - A self-checking testbench is written to apply inputs, generate clock/reset, and check results.
    - Edge cases: zero input maximum values random inputs.
3. **Simulation:**
    - Utilized EDA Playground for simulation and verification.The Commercial Simulators which are used are Synopsys VCS 2023.3,Cadence Xcelium 23.09,Aldec Riviera Pro 2023.04 and Siemens Questa 2024 and free simulators like:Icarus Verilog 12.0.
    - Techniques include simulation, formal verification, and emulation to identify bugs early in the process.
4. **Debug and Verification:**
    - Verified the functional correctness of the design and Checked corner cases and edge cases to ensure robustness.
    - Debugged and fixed any issues that arose during simulation.
5. **Logical Synthesis:**
    - In the real world design flow, the next step would be a synthesize the RTL design to a gate-level netlist.Where Yosys 0.37 is used for the generation of the RTL to Gate level netlist conversion.
    - Gate-level simulation would be performed to verify the design’s functionality and timing

2.3 SUMMARY:

In this chapter, we have given a brief about the RTL DESIGN AND VERIFICATION model. In the forthcoming chapter, we will be dealing in depth with the implementation and the result obtained from the project work.

CHAPTER 3

RESULT AND DISCUSSION

3.1 PROPOSED SYSTEM

The proposed system is a **Digital Security Locker** modelled as a Finite State Machine (FSM) in Verilog HDL. A person enters a 4-bit password on the input lines and presses a submit button. The locker compares the entered value with a stored password and either unlocks, allows another attempt, or locks the user out and sounds a buzzer after three wrong attempts. Such a controller is the core of door locks, electronic lockers, vaults and access-control panels, and an FSM is the natural way to describe it because its behaviour depends on both the present input and the history of earlier attempts.

3.1.1 Design Specification

Module: **digital_security_locker**. The stored password is a module parameter, `PASSWORD = 4'b1010`, so it can be changed at instantiation without editing the design.

Name	Type	Width	Description
clk	Input	1	Clock signal, used to synchronise the circuit
rst	Input	1	Asynchronous, active-high reset; returns the locker to a known state
submit	Input	1	Submit pulse; when high in the IDLE state the password on user_password is captured
user_password	Input	4	4-bit password entered by the user
unlock	Output	1	High when the correct password has been accepted (locker open)
buzzer	Output	1	High after three wrong attempts (alarm)
attempt_count	Output	2	Number of wrong attempts made so far (0 to 3)

Table 3.1: Digital Security Locker port specifications

FSM states. The controller uses four states with a 2-bit binary encoding, as listed in Table 3.2.

State	Encoding	Description
IDLE	2'b00	Waits for the user. When submit = 1 the entered password is latched and the FSM moves to CHECK.
CHECK	2'b01	Compares the latched password with PASSWORD. Decides between unlock, retry and lockout.
UNLOCKED	2'b10	Correct password accepted: unlock = 1, buzzer = 0. Stays here until reset.
LOCKOUT	2'b11	Three wrong attempts made: buzzer = 1, unlock = 0. Stays here until reset, even if the correct password is entered later.

Table 3.2: FSM state encoding and description

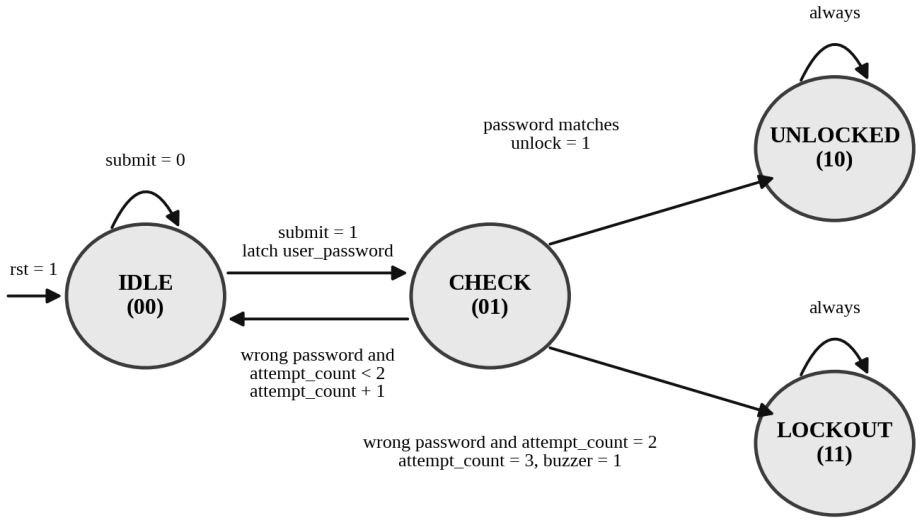


Figure 3.1: State diagram of the Digital Security Locker FSM

The complete behaviour of the FSM, including the effect on the outputs, is summarised in the state transition table below.

Present state	Condition	Next state	unlock	buzzer	attempt_count
IDLE	submit = 0	IDLE	0	hold	hold
IDLE	submit = 1	CHECK	0	hold	hold
CHECK	entered_password == PASSWORD	UNLOCKED	1	hold	hold
CHECK	mismatch and attempt_count != 2	IDLE	0	hold	+ 1
CHECK	mismatch and attempt_count == 2	LOCKOUT	0	1	3
UNLOCKED	don't care	UNLOCKED	1	0	hold
LOCKOUT	don't care	LOCKOUT	0	1	hold

Table 3.3: State transition table (hold = value is unchanged)

Hardware realisation. When the RTL is synthesised, each construct maps onto standard logic blocks:

- **Registers** – built from D flip-flops with asynchronous reset: state (2 flip-flops), entered_password (4), attempt_count (2), unlock (1) and buzzer (1), i.e. 10 flip-flops in total.
- **Comparator** (entered_password == PASSWORD) – a 4-bit equality check. Because PASSWORD is a constant, it reduces to a small AND network with inverters on the bits that are 0 in the stored password.
- **Attempt counter** (attempt_count + 1) – a 2-bit incrementer built from an XOR and an AND gate, plus a 2-input decode for attempt_count == 2.
- **Next-state and output logic** – the case statement and the if / else conditions are implemented with AND, OR, NOT gates and multiplexers.

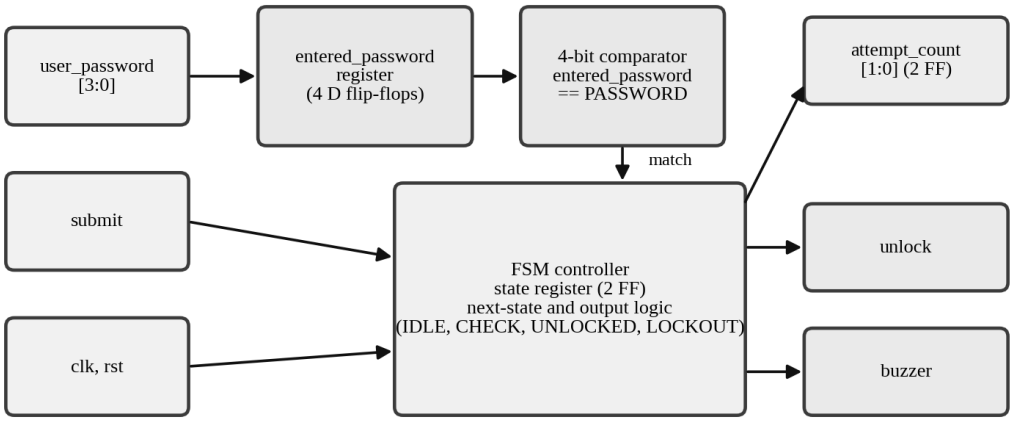


Figure 3.2: Block diagram of the Digital Security Locker

Figure 3.3 shows the gate-level circuit of the password comparator. Since PASSWORD = 4'b1010, the bits that must be 0 (bits 2 and 0) are inverted and the four lines are combined in a single AND gate, whose output goes high only for the correct password. Figure 3.4 shows the circuit of the attempt counter: the incrementer (next a0 = a0′, next a1 = a1 XOR a0) and the decoder (a1 AND a0′) that detects attempt_count == 2, i.e. the third wrong attempt.

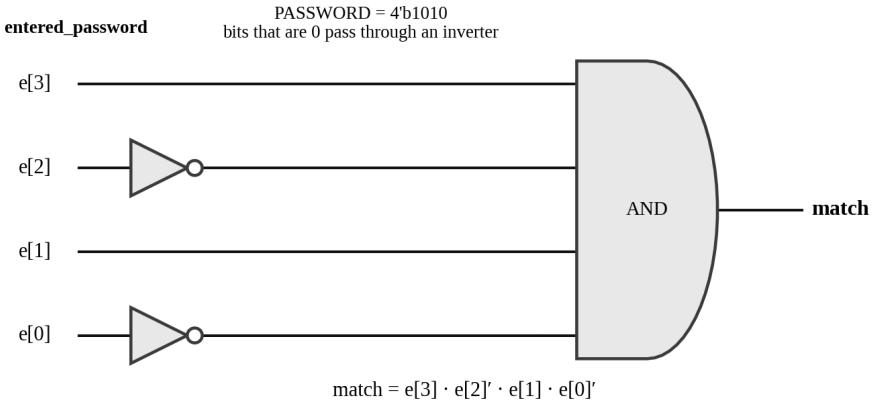


Figure 3.3: Gate-level circuit of the 4-bit password comparator (PASSWORD = 1010)

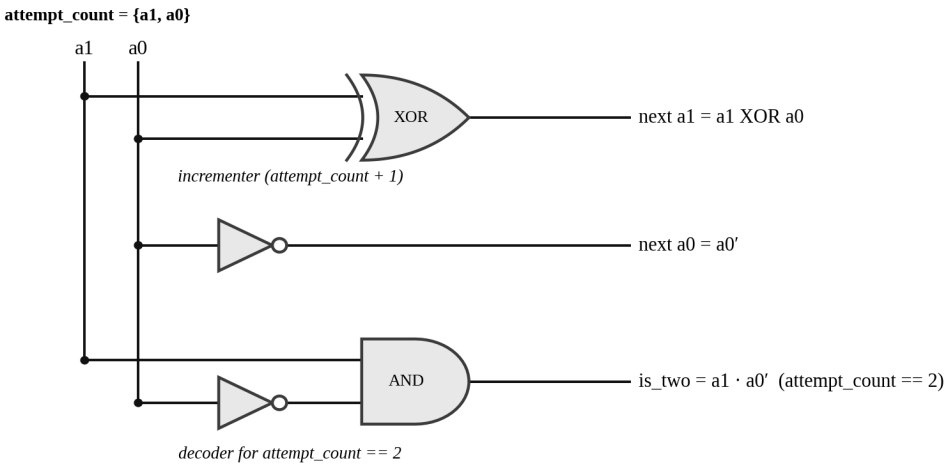


Figure 3.4: Circuit of the 2-bit attempt counter (incrementer and decoder)

3.1.2 RTL DESIGN CODE in Verilog HDL for the Digital Security Locker

The complete RTL description is given below. The FSM is written in a single clocked always block with an asynchronous active-high reset.

```
module digital_security_locker #(
    parameter [3:0] PASSWORD = 4'b1010
)(
    input wire clk,
    input wire rst,
    input wire submit,
    input wire [3:0] user_password,

    output reg unlock,
    output reg buzzer,
    output reg [1:0] attempt_count
);

    // FSM state declarations
    localparam IDLE     = 2'b00;
    localparam CHECK    = 2'b01;
    localparam UNLOCKED = 2'b10;
    localparam LOCKOUT  = 2'b11;

    reg [1:0] state, next_state;
    reg [3:0] entered_password;

    // Sequential logic: state, password and attempt counter
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state            <= IDLE;
            entered_password <= 4'b0000;
            attempt_count    <= 2'b00;
            unlock           <= 1'b0;
            buzzer           <= 1'b0;
        end
        else begin
            case (state)

                IDLE: begin
                    unlock <= 1'b0;
                    if (submit) begin
                        entered_password <= user_password;
                        state <= CHECK;
                    end
                end

                CHECK: begin
                    if (entered_password == PASSWORD) begin
                        state  <= UNLOCKED;
```

```
                    unlock <= 1'b1;
                end
                else begin
                    if (attempt_count == 2'd2) begin
                        attempt_count <= 2'd3;
                        state  <= LOCKOUT;
                        buzzer <= 1'b1;
                        unlock <= 1'b0;
                    end
                    else begin
                        attempt_count <= attempt_count + 1'b1;
                        state  <= IDLE;
                        unlock <= 1'b0;
                    end
                end
            end

            UNLOCKED: begin
                unlock <= 1'b1;
                buzzer <= 1'b0;
                state  <= UNLOCKED;
            end

            LOCKOUT: begin
                unlock <= 1'b0;
                buzzer <= 1'b1;
                state  <= LOCKOUT;
            end

            default: begin
                state  <= IDLE;
                unlock <= 1'b0;
                buzzer <= 1'b0;
            end

        endcase
    end
end

endmodule
```

Key points of the design

- **Parameterised password:** PASSWORD is a parameter, making the module reusable for any 4-bit code.
- **Asynchronous reset:** `posedge rst` in the sensitivity list returns the FSM to IDLE and clears the password register, attempt counter, unlock and buzzer.
- **Two-cycle evaluation:** the password is latched in IDLE and compared in CHECK, so the result of a submission appears two clock edges after submit is sampled.
- **Attempt limit:** attempt_count increments after each wrong password (0, 1, 2). The third wrong password sets attempt_count to 3, enters LOCKOUT and turns the buzzer on.
- **Secure lockout:** UNLOCKED and LOCKOUT are terminal states. Once locked out, even the correct password is ignored until the system is reset.
- **Safe default:** the default branch sends any illegal state back to IDLE with unlock and buzzer cleared.

3.1.3 RTL Verification

Front-End RTL Verification is the process of ensuring that a digital design, expressed at the Register Transfer Level using a hardware description language such as Verilog, meets its intended functional specification. It is a critical step in the pre-silicon design lifecycle because design flaws are found and corrected before the design is sent for physical implementation. For the Digital

Security Locker a self-checking testbench was written; it generates the clock and reset, applies four password attempts through a reusable task, and prints a PASS or FAIL message by checking the final values of the outputs.

The test plan is given in Table 3.4.

Test	Stimulus (user_password)	Expected result
Reset	rst = 1 for two clock cycles	State = IDLE; attempt_count = 0; unlock = 0; buzzer = 0
Test 1	4'b0011 (wrong)	attempt_count = 1; unlock = 0; buzzer = 0; back to IDLE
Test 2	4'b0101 (wrong)	attempt_count = 2; unlock = 0; buzzer = 0; back to IDLE
Test 3	4'b1100 (wrong)	attempt_count = 3; buzzer = 1; unlock = 0; FSM in LOCKOUT
Test 4	4'b1010 (correct, after lockout)	Locker must stay locked: unlock = 0; buzzer = 1; attempt_count = 3

Table 3.4: Test plan for the Digital Security Locker

Digital Security Locker testbench:

```
`timescale 1ns/1ps

module tb_security_locker;

    reg clk;
    reg rst;
    reg submit;
    reg [3:0] user_password;

    wire unlock;
    wire buzzer;
    wire [1:0] attempt_count;

    // Instantiate the design under test
    digital_security_locker dut (
        .clk(clk),
        .rst(rst),
        .submit(submit),
        .user_password(user_password),
        .unlock(unlock),
        .buzzer(buzzer),
        .attempt_count(attempt_count)
    );

    // Clock generation: 10 ns period
    always #5 clk = ~clk;

    // Task to submit a password
    task enter_password;
        input [3:0] password;
        begin
            @(negedge clk);
            user_password = password;
            submit = 1'b1;

            @(negedge clk);
            submit = 1'b0;

            // Wait for the FSM to process the password
            repeat (4) @(negedge clk);
        end
    endtask
```

```
    initial begin
        // Initialize signals
        clk = 1'b0;
        rst = 1'b1;
        submit = 1'b0;
        user_password = 4'b0000;

        // Generate waveform dump
        $dumpfile("locker.vcd");
        $dumpvars(0, tb_security_locker);

        // Reset the design
        repeat (2) @(negedge clk);
        rst = 1'b0;

        // Test 1: Wrong password
        enter_password(4'b0011);

        // Test 2: Wrong password
        enter_password(4'b0101);

        // Test 3: Wrong password -> buzzer ON
        enter_password(4'b1100);

        // Test 4: Correct password after lockout
        // Locker must remain locked
        enter_password(4'b1010);

        // Display results
        $display("Attempts = %d", attempt_count);
        $display("Unlock   = %b", unlock);
        $display("Buzzer   = %b", buzzer);

        if (attempt_count == 3 && buzzer == 1 && unlock == 0)
            $display("PASS: Lockout and buzzer verified.");
        else
            $display("FAIL: Check the design.");

        #20;
        $finish;
    end

endmodule
```

Testbench features

- A 10 ns clock (100 MHz) is generated with `always #5 clk = ~clk`.
- Inputs are changed on the falling clock edge (`@(negedge clk)`) so that they are stable at the rising edge and no race occurs with the design.
- The task `enter_password` drives submit high for one clock period, then waits four clock cycles for the FSM to finish processing.
- `$dumpfile` and `$dumpvars` write the VCD file that EPWave uses to display the waveform.
- The final if / else statement checks attempt_count, buzzer and unlock automatically and prints PASS or FAIL.

3.1.4 Waveform Generation using EDA PLAYGROUND:

Waveform generation is carried out to verify functional correctness before synthesis. The design and testbench were compiled and simulated on EDA Playground, and the dumped VCD file was opened in the EPWave waveform viewer.

Simulation console output:

```
Attempts = 3
Unlock   = 0
Buzzer   = 1
PASS: Lockout and buzzer verified.
```

Figure 3.5: Console output of the self-checking testbench

EPWave waveform of the Digital Security Locker:

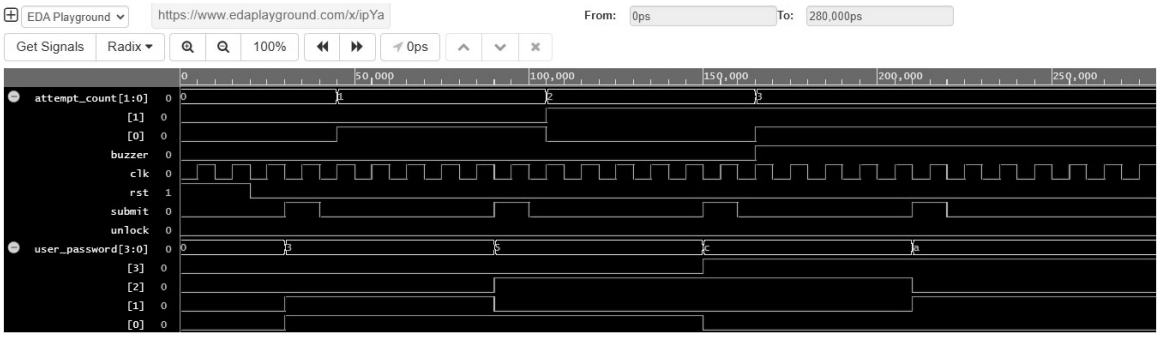


Figure 3.6: EPWave waveform generation of the Digital Security Locker

Waveform analysis. The signals shown are attempt_count, buzzer, clk, rst, submit, unlock and user_password. Reset is released at about 20 ns and the four password attempts are applied 60 ns apart. The behaviour observed in Figure 3.6 is summarised in Table 3.5.

Time (approx.)	Event	user_password	attempt_count	buzzer	unlock
0 – 20 ns	Reset active (rst = 1)	0	0	0	0
30 ns	Submit pulse, Test 1	3 (0011)	becomes 1 at 45 ns	0	0
90 ns	Submit pulse, Test 2	5 (0101)	becomes 2 at 105 ns	0	0
150 ns	Submit pulse, Test 3	c (1100)	becomes 3 at 165 ns	1 from 165 ns	0
210 ns	Submit pulse, Test 4 (correct password)	a (1010)	stays 3	stays 1	stays 0

Table 3.5: Observed behaviour from the simulation waveform

Discussion.

- After each wrong password, attempt_count rises by one, 0 → 1 → 2 → 3, exactly two clock edges after submit is sampled, which confirms the IDLE → CHECK → IDLE sequence.
- The buzzer rises together with attempt_count = 3, showing that the CHECK → LOCKOUT transition happens on the third wrong attempt.
- The unlock signal stays low for the whole simulation. In Test 4 the correct password 1010 (shown as *a* on the waveform) is applied, but the locker stays locked and the buzzer stays on, proving that LOCKOUT cannot be bypassed without a reset.
- The console message "PASS: Lockout and buzzer verified." confirms that the final values (attempt_count = 3, buzzer = 1, unlock = 0) match the specification.

Simulation procedure on EDA Playground.

1. Select Verilog as the language and a simulator from the Tools & Simulators list.
2. Paste the locker RTL into *design.sv* and the testbench into *testbench.sv*.
3. Enable *Open EPWave after run* and set the run option to dump the VCD file (the testbench already calls $dumpfile and $dumpvars).
4. Click Run, read the PASS / FAIL message in the log, then add the signals clk, rst, submit, user_password, unlock, buzzer and attempt_count in EPWave and note the transitions.

3.1.5 Operation Flow, Timing and Coverage Analysis

Operational flow. Figure 3.7 gives the flow chart of the locker. After reset the FSM waits in IDLE for the submit pulse. The submitted password is latched and compared in the CHECK state. A match leads to the UNLOCKED state, while a mismatch either returns the FSM to IDLE with attempt_count incremented, or, if two wrong attempts were already made, moves it to the LOCKOUT state.

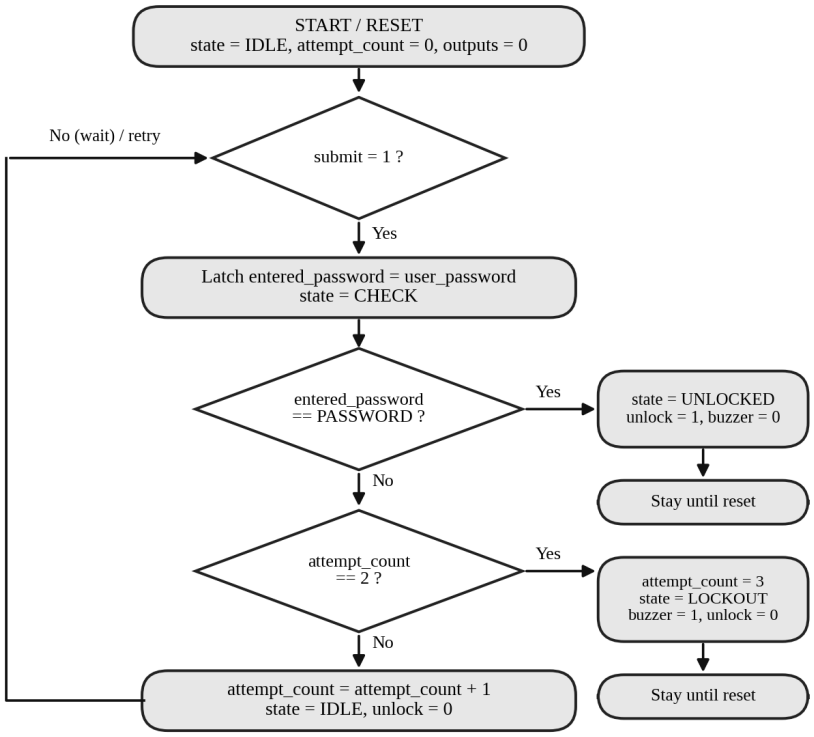


Figure 3.7: Flow chart of the Digital Security Locker operation

Timing of one submission. Figure 3.8 shows the expected timing of a single wrong password. The submit pulse is sampled at clock edge E1, which latches the password and moves the FSM from IDLE to CHECK. At the next edge E2 the comparison fails, so attempt_count is incremented and the FSM returns to IDLE. A submission is therefore evaluated within two clock cycles; this is why the testbench waits four cycles after every attempt, which leaves a safe margin before the next password is applied.

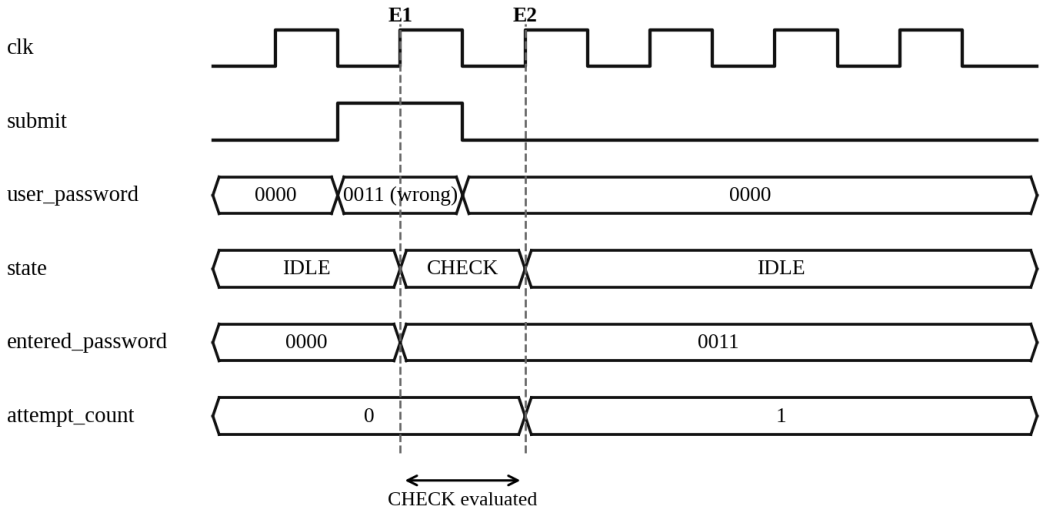


Figure 3.8: Timing diagram of one wrong-password submission

Functional coverage. Table 3.6 lists the states and transitions of the FSM and shows which of them were exercised by the testbench and visible in the waveform.

FSM item	Exercised by testbench?	Evidence
Reset to IDLE	Yes	rst = 1 at the start; all outputs low
IDLE → CHECK (submit = 1)	Yes	Four submit pulses at 30, 90, 150 and 210 ns
CHECK → IDLE (wrong, attempts < 2)	Yes	attempt_count 0 → 1 and 1 → 2
CHECK → LOCKOUT (third wrong attempt)	Yes	attempt_count = 3 and buzzer = 1 at 165 ns
LOCKOUT hold (correct password ignored)	Yes	Test 4: unlock stays 0, buzzer stays 1
CHECK → UNLOCKED (correct password)	Not in this run	Proposed as an extra test in the future scope

Table 3.6: State and transition coverage of the testbench

Estimated hardware resources. From the RTL structure, the resources required by the design are estimated in Table 3.7. These are counted from the code, not taken from a synthesis report.

Block	Implementation	Estimate
Registers	state (2), entered_password (4), attempt_count (2), unlock (1), buzzer (1)	10 D flip-flops
Password comparator	Two inverters and one 4-input AND (constant PASSWORD)	about 3 gates
Attempt counter	2-bit incrementer (inverter and XOR) and a == 2 decoder (inverter and AND)	about 4 gates
Next-state / output logic	State decode, multiplexers for the case branches	small control logic

Table 3.7: Estimated hardware resources

Proposed additional test scenarios. To close the coverage gap in Table 3.6, the following directed tests are recommended for the next verification cycle (Table 3.8).

Scenario	Stimulus	Expected result
Correct password first	1010 on the first attempt	unlock = 1, buzzer = 0, attempt_count = 0, state = UNLOCKED
Correct password on the second attempt	0011 then 1010	attempt_count = 1, then unlock = 1
Correct password on the third attempt	0011, 0101, then 1010	attempt_count = 2, then unlock = 1; buzzer stays 0
Reset after lockout	rst = 1 after Test 3	State = IDLE; attempt_count = 0; buzzer = 0; correct password is now accepted
No submit	Change user_password without submit	No state or output change

Table 3.8: Proposed additional test scenarios

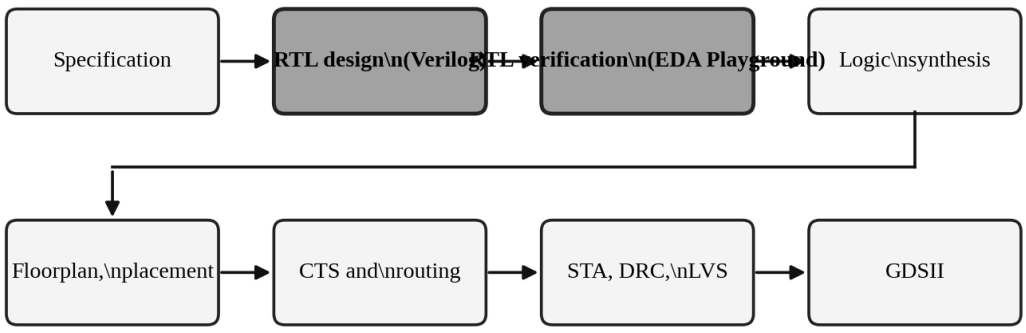
Applications.

- Electronic door locks, lockers and safes that use a keypad code.
- Access-control panels in offices, laboratories and server rooms.
- Vehicle and equipment security systems with an alarm output.
- A teaching example of a Moore/Mealy-style controller with an attempt counter and alarm.

Advantages. The design is small (10 flip-flops), fully synchronous apart from the reset, parameterised for any 4-bit password, and the terminal LOCKOUT state prevents repeated guessing. **Limitations.** The password is a fixed 4-bit parameter, the locker can be re-armed only by a full reset, and the submit input is assumed to be a clean, synchronised pulse; a real keypad would need debouncing.

3.1.6 Position of the Project in the RTL-to-GDSII Flow

The work reported here covers the front-end of the digital design flow: specification, RTL coding and functional verification. Figure 3.9 shows where these stages sit in the complete RTL-to-GDSII flow. The verified RTL is the golden reference for every later stage.



Shaded blocks: stages completed in this project

Figure 3.9: Stages of the RTL-to-GDSII flow (shaded: completed in this project)

Synthesis readiness. The locker is written in synthesisable Verilog: all state is held in flip-flops, there are no delays or initial blocks in the design, the reset is explicit, and the case statement has a default branch so that no unintended latches are inferred. The declared but unused `next_state` register has no effect on behaviour and is removed by synthesis.

Reset strategy. The asynchronous reset gives a known starting state independent of the clock, which is important for a security device that must power up locked. Reset de-assertion should be synchronised to the clock in a physical implementation to avoid recovery and removal timing violations.

Next stages. After functional verification, the RTL would be synthesised to a gate-level netlist under area, power and timing constraints, followed by floorplanning, placement, clock tree synthesis, routing, static timing analysis, DRC and LVS to produce the final GDSII layout.

3.2 SUMMARY

In this chapter, the Digital Security Locker was specified, designed as a four-state FSM in Verilog HDL and verified with a self-checking testbench on EDA Playground. The waveforms show correct reset behaviour, correct counting of wrong attempts, the buzzer alarm after the third wrong attempt, and a permanent lockout that rejects even the correct password until reset. The results satisfy the design specification at the RTL level.

CHAPTER 4

CONCLUSION AND FUTURE SCOPE

The RTL design and verification of a Digital Security Locker using a Finite State Machine was successfully completed in this training program. The locker was described in Verilog HDL as a four-state FSM (IDLE, CHECK, UNLOCKED and LOCKOUT) with a parameterised 4-bit password, a 2-bit wrong-attempt counter, an unlock output and a buzzer output. The design is compact, using only 10 flip-flops together with a 4-bit comparator, a 2-bit incrementer and small control logic.

A self-checking testbench with a reusable stimulus task was developed and simulated on EDA Playground. The EPWave waveforms confirmed that attempt_count advances from 0 to 3 on three successive wrong passwords, that the buzzer turns on at the third wrong attempt, and that the locker remains locked, with unlock = 0, when the correct password is entered after lockout. The testbench reported PASS, so the RTL meets the specified behaviour. This exercise gave hands-on experience in FSM modelling, testbench writing, waveform debugging and the front-end of the ASIC design flow.

Future Scope

The verification can be extended with a test in which the correct password is entered on the first, second or third attempt to exercise the UNLOCKED path, with random and corner-case password testing, and with SystemVerilog assertions or a UVM environment for higher coverage.

The design itself can be enhanced with a longer or user-programmable password, a debouncer and synchroniser for the submit input from a physical keypad, an automatic re-lock or timed lockout

instead of requiring a full reset, and a display or non-volatile memory to store the password.

The verified RTL can be taken forward through logic synthesis, floorplanning, placement, clock tree synthesis, routing, static timing analysis, DRC and LVS to a GDSII layout, and prototyped on an FPGA before fabrication.

This training program reinforced key concepts of RTL design and verification and established a strong foundation for tackling more complex digital design projects.

REFERENCES

1. M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Pearson Education, 2013.
2. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.
3. Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed., McGraw-Hill Education, 2014.
4. Clifford E. Cummings, "Coding and Scripting Techniques for FSM Designs with Synthesis-Optimized, Glitch-Free Outputs," *Synopsys Users Group (SNUG) Conference*, Boston, 2000.
5. IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.
6. Doulos, "EDA Playground," [Online]. Available: https://www.edaplayground.com